

第 2 课 · 让调查过程可控

沿 GPU assert 案例阅读状态、预算、错误回喂与恢复边界

从概念、项目代码到运行证据的连续阅读材料。

目录

第 2 课 • 让调查过程可控	4
2.1 本章解决的问题	4
2.2 贯穿案例：同一组观察，三种调查结局	4
先把事实与调查进度分开	4
2.3 显式状态：把一次调查变成可检查的数据	5
phase 是阶段，state 是调查记录	5
当前代码的六个阶段	5
2.4 动态计划：阶段稳定，调查选择可以改变	7
计划不是固定流水线	7
一个 advance 究竟做多少工作	7
2.5 预算：在程序中给灵活性设上限	8
三种上限，各管一个维度	8
为什么执行前后都要检查	9
token 预算不等于上下文窗口	10
2.6 停止条件：结束也必须有证据	10
成功结束与被迫停止	10
报告只有一次修正机会	11
不完整响应不能执行	11
2.7 错误回喂：让模型有机会修正，但不给它绕路	11
工具失败是结果的一种	11
编造参数：不静默替模型纠错	12
不同错误有不同终点	13
2.8 重复调用、幂等与有限重试	13
识别同一件读取，不依赖供应商的 call ID	13
幂等不是‘可以无限重试’	14
模型网关的重试已经实现，但也有界限	14
2.9 上下文与证据索引：保留什么，不能混淆什么	15
业务状态与会话运行信息	15

当前实现怎样约束上下文 15

2.10 保存、失败与恢复：当前能做到哪一步 16

 保存快照，不保留无限活跃上下文 16

 恢复页面与恢复执行不同 16

 上游失败以后怎么处理 16

2.11 真实运行证据：对照三次预算实验 17

 运行方法与来源 17

 先预测，再用轨迹解释 18

 错误反例如何核对 19

2.12 常见错误与本章总结 19

 从失败模式倒推保护位置 19

 课后阅读与模仿方向 20

 思考题与校对线索 20

第 2 课 · 让调查过程可控

沿 GPU assert 案例阅读状态、预算、错误回喂与恢复边界

2.1 本章解决的问题

上一章走通了模型选择工具、程序返回观察、模型生成报告的最小循环。接下来要回答一个更实际的问题：如果模型重复读取同一份日志、把任务编号写错、一直提出新计划，程序还能否按时停止，并解释已经发生的事情？一个循环能运行，只说明它有通路；让每次推进都有边界，才使调查可管理。

本章以 case-gpu-assert 为主线，把观察和控制分开阅读。观察回答‘发生了什么’，控制回答‘现在允许做什么、已经用了多少资源、下一轮是否还能开始’。读者会看到 Python 字典、dataclass、枚举与异常处理在系统中的具体职责，不需要先学某个 Agent 框架。

读完后，应能解释一次 model decision 为什么不等于一次工具调用；为什么返回 JSON 不一定意味着调查结束；为什么提示词中的‘不要重复’还需要程序检查；以及保存了 state 为什么仍不足以在服务器重启后续跑。课堂可以沿书中源码和已有命令带读，课后再复现结果与模仿控制设计。

本章描述当前 V2 实现。设计规格中的自动压缩上下文、持久恢复、远程工具超时等方向，不能被当成已经实现的功能。真实 API 是主路径；书中的固定测量使用人工编写回放（authored replay）。

2.2 贯穿案例：同一组观察，三种调查结局

先把事实与调查进度分开

任务 job-v2-101 在四个 rank 上运行。rank 2 在 UTC 01:01:00 出现 device-side assert 与索引断言；rank 0、1、3 在约十分钟后报告集体通信超时。采样显存与完整性说明为排查提供另一类观察。这些素材来自教学案例文件，不会因为我们把调查预算改为两步就改变。改变的是系统来得及读到什么，以及它是否获得了一份通过校验的报告。

源码定位：data/v2/cases/case-gpu-assert.json

默认回放用五轮模型响应完成调查：计划、三工具读取、Runbook 检索、假设、报告。若只允许两轮，程序仍会读完第二轮中的背景、日志、指标，随后停在三份证据；若 token 总预算只有 200，第一轮响应的预置 usage 已经达到限额，连计划控制对象也不再处理。同一个故障案例可以对应完成、部分调查和几乎没有观察三种结果。

条件	观察到的终点	能够声明什么
默认预算	completed；四份证据；合法报告	本次调查发布了有引用的诊断
max_steps=2	stopped；三份证据；无报告	已收集部分事实，未发布诊断
max_tokens=200	stopped；零份证据；无报告	预算在首轮响应后耗尽

这张表避免一种误读：不能根据自己已经看过的完整教材结论，把有限步数的调查也标为成功。报告是否存在、证据是否收集、会话是否允许继续，是三个不同问题。控制层不会在停止时偷偷补一个 `gpu_device_assert` 结论，也不会切换成规则推理器。后文会用真实 CLI 对这三个终点核对。

2.3 显式状态：把一次调查变成可检查的数据

phase 是阶段，state 是调查记录

术语 · 调查状态 (state)
一次会话当前可序列化的业务记录，包括目标、阶段、预算、计划、假设、证据索引、报告和停止原因。

如果仅用一个 `messages` 列表表示调查，界面和服务端就得从自然语言中猜测 ‘已经完成了吗’。V2 使用 `InvestigationState`，把运行事实放在明确字段中。phase 是其中的阶段值，不是整份状态，也不是模型的自由文本。业务方可以检查 phase 与 report，而不必依赖答案是否以 ‘综上’ 结尾。

字段	职责	阅读时的边界
session_id / case_id / question / mode	会话身份、案例、目标和模型模式	不同会话证据隔离；mode 区分 online 与 replay
phase / stop_reason	当前阶段与停止原因	stopped 不自动变成 completed
budget	上限与已用资源	包含 max_steps、max_tokens、deadline_at 等
plan	模型给出的简短调查计划	是说明与可更新记录，不是硬编码调度表
hypotheses	候选假设及支持、反驳引用	支持多个；本案例回放只记录一个
evidence_ids	本会话观察到的证据索引	原文在 Evidence Store；这里保存标识
report	通过校验后发布的报告	无合法报告时通常为 None

源码定位：src/ailab_ops/investigation/models.py · InvestigationState

状态的 `to_dict` / `from_dict` 负责将枚举、日期、嵌套对象转成 JSON 可处理的结构，并能还原业务记录。这使测试、接口展示和外部持久化有统一形状。但是，序列化只是保存形式；它不等于服务器已经接入数据库，也不等于把 JSON 还原后就能续跑。执行所需的会话上下文在另一个对象中。

当前代码的六个阶段

源码摘录：阶段名称以枚举为准

```
class InvestigationPhase(str, Enum):
    INTAKE = "intake"
    INVESTIGATING = "investigating"
    VALIDATING = "validating"
    AWAITING_APPROVAL = "awaiting_approval"
    COMPLETED = "completed"
    STOPPED = "stopped"
```

源码定位：src/ailab_ops/investigation/models.py · InvestigationPhase

阶段	进入原因	后续边界
intake	start 创建会话并保存初始状态	检查预算后才开始模型决策
investigating	advance 请求一轮模型响应	处理计划、工具、假设或报告
validating	校验报告或等待唯一一次报告修正	合格则完成；不合格则有限修正
completed	合法报告；或审批流程结束	读循环已终止；可另行申请动作
awaiting_approval	合法诊断后创建待审批动作	读循环不推进；审批流程接管
stopped	耗尽预算、取消、模型失败等	保留事实与原因；不发布替代猜测

‘获取背景’、‘收集日志’和‘形成假设’是理解调查的概念步骤，当前实现没有把它们各自定义为 phase。它们都可发生在 investigating。代码也没有一张独立的全量状态迁移矩阵；阶段赋值分散在 start、advance、_report、_stop 和审批回调中。我们依据这些真实分支讲状态机，不另造状态名。



主调查路径；预算、取消或后端失败可进入 stopped，首次报告校验失败可再请求一次修正

图中是主要成功路径，不能被读成只能单向推进。首次报告校验失败时保存 validating，下一次 advance 又先设置 investigating；但 report_failures 会限制这一轮只能修正报告。completed 之后，调用者可以显式 request_action，进入 awaiting_approval，审批执行、拒绝、过期或失败后回到 completed。动作结局不抹掉已经完成的诊断；具体审批约束在第 4 课展开。

章节关联：第 4 课 · 动作建议、人工审批与模拟执行

2.4 动态计划：阶段稳定，调查选择可以改变

计划不是固定流水线

模型可返回 `{type: plan, plan: [...]}`。 `parse_control` 按 `type` 解析对象， `_control` 把 `plan` 列表整体写入 `state.plan`。模型后续再提交 `plan`，会替换之前的计划；这里没有逐项完成标记，也没有自动把文字映射成函数执行。真正的读取仍由 `native tool_calls` 发起，经过工具、参数与策略检查。

源码摘录：更新计划记录，不按文字执行函数

```
if isinstance(control, PlanControl):
    state.plan = control.plan
    self._save(session, "plan_updated")
```

源码定位： `src/ailab_ops/investigation/orchestrator.py` · `_control`

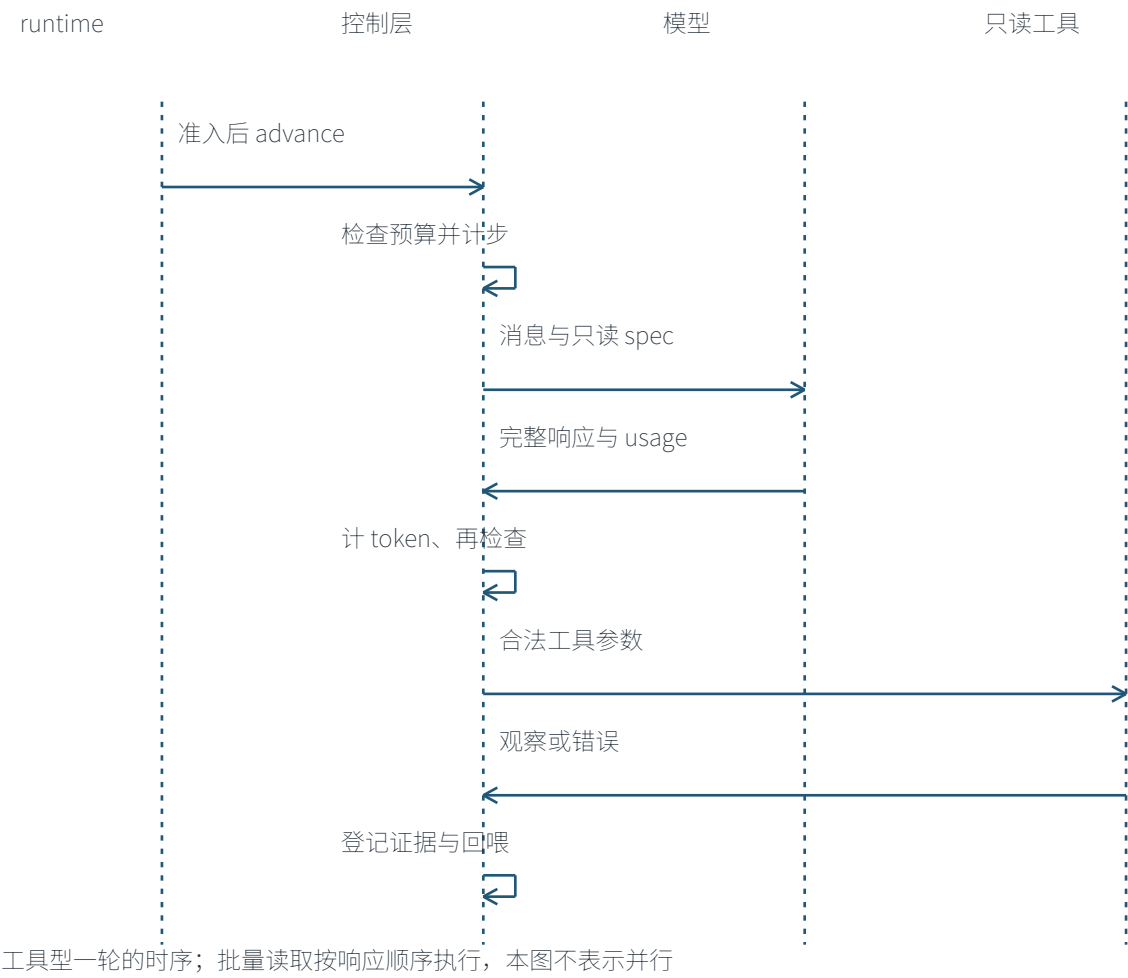
例如，读日志前可以计划比较全部 `rank` 的起始错误。读到本地索引断言后，计划可以转向查询 `device assertion Runbook` 与核查采样显存，避免只围绕末尾 `NCCL timeout` 转圈。这是合理的在线调查推演；本章默认 `authored replay` 只在第一轮提交一次计划，并未录制计划改写。能够更新字段，与本条回放展示了更新，是两件事。

状态机负责约束每一轮的入口、结束和动作权限，不替模型固定每个工具的顺序。若模型先读 `metrics`，程序并不会强制先读 `logs`；假设列表也由模型提出。确定性测试用不同脚本化模型选择 `read_logs` 或 `read_metrics`，验证程序接受不同合法路径。脚本化边界验证的是控制程序，不代表真实模型一定会选择好路径。

源码定位： `tests/test_orchestrator.py`

一个 advance 究竟做多少工作

`advance` 先确认传入状态属于一个 `live session`，再检查取消与预算，设置 `investigating`，并将 `steps_used` 加一。随后请求模型，累计 `usage`，判断响应完整性，再分流到工具请求或控制对象。如果响应没有工具调用，也可能只是更新计划或假设，不能把‘无工具’当成‘最终答案’。



回放第二轮含三个工具请求，所以一轮决策内会连续处理三个读取。计划轮、假设轮和报告轮同样消耗 step，却没有工具执行。理解计数单位后，才能解释五个 step、四次工具调用的完整调查。若要额外限制单轮工具数量，需增加相应约束；max_steps 本身没有承担这个职责。

2.5 预算：在程序中给灵活性设上限

三种上限，各管一个维度

max_steps 限制模型决策轮数，max_tokens 限制已返回响应累计的输入与输出 token，deadline_at 是绝对截止时间。steps_used、tokens_used 是计数，不是模型估计的自我报告。runtime 接收 deadline_s，并以当前 UTC 时间加秒数构造 deadline_at；Budget 自身也允许不设日期，但当前 runtime 调查会提供截止时间。

源码摘录：等于上限时也算耗尽，可能同时命中多个限制

```
def exhausted(self, now: datetime) -> list[str]:
    """Return every reached limit in steps, tokens, deadline order."""
    limits = []
    if self.steps_used >= self.max_steps:
        limits.append("steps")
    if self.tokens_used >= self.max_tokens:
        limits.append("tokens")
    if self.deadline_at is not None and now >= self.deadline_at:
        limits.append("deadline")
    return limits
```

源码定位：src/ailab_ops/investigation/models.py · Budget

exhausted 返回命中限制的列表，因此 stop_reason 可为 budget_exhausted:steps,tokens 等组合，不必只保留最先检查的一项。达到上限时，控制层保存 stopped 与原因。已经收集的证据保留，但不能据此伪造一份报告；stop_reason 是控制事实，不是故障根因。

为什么执行前后都要检查

一轮开始时预算未耗尽，不保证模型响应回来时仍有时间。模型可能慢，返回 usage 也可能用尽 token；一个批次中的第一个工具也可能耗时。控制层在模型调用前检查全部预算，返回后和每次工具前后检查 token、deadline 与取消，最后再检查步骤上限。检查位置决定哪些观察能进入状态。

本轮 steps_used 在请求模型前增加。即使网关失败，这轮也已开始并被计数。但一轮内部 include_steps=False，允许最后一轮已请求的合法工具读完；否则每次刚把计数加到上限，就会拒绝处理这一轮的有效响应。step 上限阻止下一轮，时间、token 和取消则可阻止本轮后续读取。

源码摘录：轮内不以 step 上限截掉已批准的本轮工作

```
limits = session.state.budget.exhausted(self._now())
if not include_steps:
    limits = [limit for limit in limits if limit != "steps"]
if limits:
    self._stop(session, "budget_exhausted:" + ",".join(limits))
return bool(limits)
```

源码定位：src/ailab_ops/investigation/orchestrator.py · _stop_if_exhausted

如果 deadline 在第一个同步工具执行期间到达，当前工具无法被此检查函数强制中断。它返回的有效观察先被登记，随后停止后续工具。若 deadline 在模型响应期间到达，则响应后的检查可以在工具执行前停止。远程工具若可能长时间阻塞，需要在工具适配层实现超时和取消，不能仅依赖循环外的日期比较。当前案例工具只在内存中读取教学数据。

源码定位：tests/test_orchestrator.py

token 预算不等于上下文窗口

源码摘录：输出请求上限取 4096 与剩余累计预算的较小值

```
response = self.gateway.complete(deepcopy(session.messages),
    [self.registry.get(name).spec() for name in self.registry.names
    if self.registry.get(name).kind == "read"],
    max_tokens=min(4096, state.budget.max_tokens - state.budget.tokens_used))
```

源码定位：src/ailab_ops/investigation/orchestrator.py · advance

这里的请求 max_tokens 主要约束模型输出；输入消息也会消耗 token，而上一轮已用量并不预先知道下一轮的完整输入计费。因此 max_tokens 是检查累计 usage 后停止的预算，并非供应商级别绝不会超额的硬保证。模型网关缺失 usage 时会使用近似计数；回放则使用条目预置值。计数用于控制，测量精度仍取决于来源。

runtime 在准入前估算消息 token 加输出上限，检查租户用量与费用，并在请求返回后结算。这些服务级限制与会话 Budget 配合，但不能混称为同一个字段。Budget 没有 max_cost 字段，也没有独立 context_window 字段；成本配置、并发和限流在第 6 课阅读。

源码定位：src/ailab_ops/v2_runtime.py · _advance

章节关联：第 6 课 · 服务准入、租户用量与模型调用控制

2.6 停止条件：结束也必须有证据

成功结束与被迫停止

模型想结束时，应提交 report 控制对象。程序先验证结构，再校验重要主张是否有引用、引用是否存在且属于本会话。通过后才设置 report 与 completed。截断与缺失需模型表达限制，当前校验器并不自动判断这一点。‘模型没有再调工具’、‘回答很长’、‘置信度很高’都没有独立结束权限。引用结构校验仍不证明根因在语义上正确。

源码定位：src/ailab_ops/investigation/orchestrator.py · _report

停止或交接条件	真实状态或原因	结果处理
合法报告	completed	保存报告与证据；释放活跃读会话
预算耗尽	stopped / budget_exhausted:...	保留已收集事实；没有额外诊断回退
取消	stopped / cancelled	协作取消后禁止新模型与后续读取
模型后端失败	stopped / model_backend:<kind>	保留部分调查与分类错误
报告修正仍失败	stopped / report_validation_failed	阻止发布不合法报告
有效动作申请	awaiting_approval	读循环停止，独立审批流程负责后续

报告只有一次修正机会

首次报告不合法，`_report_issues` 累加 `report_failures`，回喂问题并让模型重新生成一次。下一轮若仍不是合法报告，就 `stopped`。模型不能在这个修正机会里再次读工具、更新计划，或者用不完整响应获得更多机会。预算也不会为了修正而放宽：若下一轮开始前已耗尽，就直接停止。

源码摘录：报告失败计数与修正阶段；随后回喂 `issues`

```
session.report_failures += 1
if session.report_failures > 1:
    self._stop(session, "report_validation_failed")
else:
    session.state.phase = InvestigationPhase.VALIDATING
```

源码定位：src/ailab_ops/investigation/orchestrator.py • `_report_issues`

反例是模型编造 `ev-forged` 引用，再不停申请新日志来逃避报告校验。当前代码会把第一次未知引用作为 `issues` 回喂，只接受一次报告修正；修正轮请求工具直接导致 `report_validation_failed`。如果调查确实需要更多观察，应在报告前完成，或由用户开启新的调查，而不是在校验失败后无限扩张任务。

源码定位：tests/test_orchestrator.py

不完整响应不能执行

流式模型响应可能因为长度限制结束，或者带 `error` 结束，即便缓冲区里已经拼出部分工具参数，也不能执行。`advance` 对 `finish_reason=length/error` 回喂 `incomplete_response`，并等待预算内下一轮。不完整的助手工具调用不加入历史，避免下一次请求里出现没有匹配 `tool` 结果的悬空调用。若发生在报告修正轮，则直接停止，不再多给一次机会。

源码定位：tests/test_orchestrator.py

另一种情况是 SSE 流尚未收到终止 `finish_reason` 就断开。网关把它当 `protocol` 后端错误，而不是把已经读到的碎片假装完整答案。这类错误会走模型失败路径。完整性判断发生在模型网关与控制层两处；它们保护不同边界：前者确认协议结束，后者确认响应可执行。

2.7 错误回喂：让模型有机会修正，但不给它绕路

工具失败是结果的一种

工具调用不是只能返回成功数据。`ToolResult` 有 `ok`、`data`、`error`、`hint`、`truncated` 等字段；失败结果不登记成证据。`_tool_feedback` 将结果变成 `role=tool` 的消息，保留 `name` 与 `tool_call_id`，让模型知道哪次请求出了问题。真实工具异常由注册工具包装为失败结果，因此控制循环通常有机会继续。

阅读示意：_tool_feedback 原调用表达式重新分行，参数与行为不变

```
session.messages.append(  
    ChatMessage(  
        "tool",  
        json.dumps(payload, ensure_ascii=False, default=str),  
        name=call.name, tool_call_id=call.id,  
    )  
)  
self._save(session, "tool_result")
```

源码定位：src/ailab_ops/investigation/orchestrator.py · _tool_feedback

与此不同，JSON 控制对象格式不对、假设引用未知证据等错误使用 _feedback，以 user 消息写入 error、hint 和必要 issues。两者都让错误成为下一轮可见输入，却保留工具结果与控制反馈的区别。‘可修正’仅意味着模型有机会；模型是否修正对，仍要由下一轮实际输入和校验结果证明。

编造参数：不静默替模型纠错

案例工具 get_case_logs 只接受 case_id，schema 把它限定为当前案例 ID，且 additionalProperties=False。如果模型传 {job_id: job-v2-101}、把 case_id 写成不存在的值，或擅自加入 node_id，策略授权与参数检查会拒绝。即使人能猜到它想做什么，程序也不偷偷改字段，否则审计中模型请求与实际执行会不一致。

结构示例：缺少 case_id 的工具错误回喂；不是默认回放中的额外一轮

```
{"ok": false,  
 "error": "invalid arguments: arguments.case_id is required",  
 "hint": "Repair the arguments using the tool schema; actions must be proposed."}
```

源码定位：src/ailab_ops/investigation/parsing.py · validate_arguments

validate_arguments 检查本项目使用的 schema 词汇，如类型、必需项、枚举、额外字段、数值范围和字符串长度。它不是完整 JSON Schema 标准验证器，未知关键词不会自动形成保护。策略还负责工具类型与授权。通过格式校验也不意味着任何参数都符合业务目的，接入企业工具仍需增加相应资源和身份约束。

源码定位：src/ailab_ops/policy/engine.py · authorize

不同错误有不同终点

错误	回喂或停止	下一步责任
未知工具 / 参数错误	tool 消息 ok=false；无证据	模型选择已注册工具或修正参数
工具返回失败 / 无效证据	tool 消息失败；无证据增量	模型选择其他读取，或在预算内再试
重复成功读取	duplicate_call 与已有 evidence_ids	使用已有证据或调整合法查询
普通控制 JSON 不合法	user 消息 invalid_control	下一轮重写；总预算仍生效
报告问题	report_validation；仅一次修正	基于已有证据修正报告
模型 timeout / rate_limited 超限	model_backend:<kind>; 停止	用户或服务方检查上游，再开调查

普通工具参数错误没有专门的‘只允许一次’计数；当前代码靠总预算限制修正次数。工具失败也不会自动进入网关的 HTTP 重试循环。报告修正则有明确的一次上限。把三类失败统称为‘失败后重试一次’，会失去当前实现最关键的差异。

2.8 重复调用、幂等与有限重试

识别同一件读取，不依赖供应商的 call ID

模型可能用不同 call_id 请求同一个工具和相同参数。call_id 关联一次协议消息，不能说明读取是否新鲜。_read_tool 将工具名和排序后的参数编码成键，成功捕获证据后记录 completed_calls。同一会话再次命中键，就返回 duplicate_call 和已有证据 ID，不重新执行工具。

源码摘录：按工具与规范化参数阻止重复成功读取

```
key = json.dumps([call.name, arguments], sort_keys=True, ensure_ascii=False, allow_nan=False)
if key in session.completed_calls:
    self._tool_feedback(session, call, {"ok": False, "error": "duplicate_call",
    "hint": "Use the existing evidence or change the query.",
    "evidence_ids": session.completed_calls[key]})
return
```

源码定位：src/ailab_ops/investigation/orchestrator.py · _read_tool

参数字典键顺序变化不会避开这个检查；工具名、参数值不同则是另一次请求。这不是语义去重：两个略有差异却意图相同的检索 query 可能仍被执行。也不是生产环境的刷新缓存：目前案例世界固定，成功后重复读取被拒绝；若真实指标随时间变化，应设计显式时间窗口或刷新语义，而不能默认把任何重读都当成浪费。

反例是读完 logs 后模型说‘为了确认，再读一次’，但参数完全相同。第二次不产生新事实，控制层返回已有 ID；若模型仍反复请求，轮数和 token 继续消耗，最终预算停止。重复检测减少冗余 I/O，却不能替代总预算，因为模型仍可能不断产生重复请求本身。

源码定位：tests/test_orchestrator.py

幂等不是 ‘可以无限重试’

术语 · 幂等 (idempotent)
重复提交相同操作不会因为提交次数增加而产生额外副作用。读取通常符合这一点，但返回值可能随时间变化。

Tool 元数据包含 kind、sensitivity、idempotent。只读工具默认 idempotent=True；当前动作工具只走审批后的模拟执行。这个标记用于说明调用性质，不代表 orchestrator 已实现通用幂等重试器。失败调用不加入 completed_calls，因此模型可以在后续轮次再次请求；成功调用才进入重复检测。

源码定位：src/ailab_ops/tools/registry.py · Tool

接入远程工具后，若查询遇到临时网络错误，可以在总期限内有限重试。但写动作遇到断连时，服务端可能已经执行，客户端却没收到结果；简单重试可能重复重启任务。真正的动作幂等需服务端幂等键、执行记录与状态查询配合。教学模拟审批与只读重复键不能被宣传为这些机制已经齐备。

模型网关的重试已经实现，但也有界限

OpenAIModelGateway 对临时 transport/timeout、普通 429 rate_limited、408 与部分 5xx 错误作有限重试。默认 max_retries=2 意味着一次 complete 最多三次 HTTP 尝试；这些尝试仍属于同一个模型 step，不会把 steps_used 加三。未拿到合法响应的上游尝试也可能已经耗费供应商资源，没有返回 usage 时本地无法据此得出精确账单。

失败类型	当前处理	为何不能都重试
普通 429 / 408 / 500、502、503、504	可重试；尝试次数有限	临时拥堵或服务异常可能恢复
429 且错误正文指向 quota	quota；不可重试	额度不足通常不会因等几秒解决
401 / 403 / 404 / 其他 4xx	对应分类；不可重试	需要修正认证、权限或请求配置
格式不合法的模型响应	protocol；不可重试	不能将坏协议当已完成响应

源码定位：src/ailab_ops/llm/openai_compat.py · classify_status

后端提供有效 Retry-After 时，网关优先采用；否则使用带随机抖动的指数退避。retry_max_delay_s 默认 30 秒，将异常大的等待建议截到有限范围，非法或非有限数值也不会直接拿来等待。若本轮已取消或截止，ModelRequestControl 会阻止退避后的下一次请求。上游建议并不拥有延长调查期限的权限。

源码摘录：统一规范化上游等待建议与本地退避

```
hint = error.retry_after_s
if hint is None or not math.isfinite(hint) or hint < 0:
    hint = self._backoff(attempt + 1) if error.retryable else None
if hint is not None:
    hint = min(hint, self.retry_max_delay_s) if math.isfinite(hint) and hint >= 0 else 0.0
```

源码定位：src/ailab_ops/models/openai.py · complete_with_control

反例是上游连续限流，程序一边循环 sleep 一边重试，不检查总截止时间。用户放弃后，服务仍持续请求。当前 runtime 通过 GuardedGateway 把取消 Event 和 deadline_at 传入网关，每次尝试前重查，等待时也

可被取消打断。若直接调用 `gateway.complete` 而没有 `control`，只有网关自身超时和重试上限，不具有整个 `runtime` 的调查期限保护。

源码定位：`src/ailab_ops/models/protocol.py · ModelRequestControl`

HTTP 连接/读写超时与绝对截止时间不是同一种机制。HTTPX 超时限制 I/O 等待；控制对象在尝试、退避等边界复查总期限。同步在途工作仍需协作结束，不能把这些检查说成任意时刻强制杀掉全部工作。

2.9 上下文与证据索引：保留什么，不能混淆什么

业务状态与会话运行信息

`InvestigationSession` 包含 `state`、`evidence_store`、`messages`、`events`、`completed_calls`、`report_failures`，以及动作申请信息。`state` 用来展示与序列化；`messages` 用来请求下一轮模型；`completed_calls` 用来去重；`report_failures` 用来限制修正次数。只存 `state` 会丢失后几项的执行语义。

源码定位：`src/ailab_ops/investigation/orchestrator.py · InvestigationSession`

一次调查启动时，消息包括 `system` 协议与 `user` 目标及动作说明。工具读取后，`tool` 消息带入实际数据和 `evidence_items`；报告引用的 `evidence_ids` 指向 Evidence Store。状态中的证据索引适合审计与展示，但模型判断还需要相关内容，不是给一个长 ID 就自动知道其原文。

信息载体	包含什么	不能代替什么
<code>state.evidence_ids</code>	本会话已观察证据的 ID 列表	不等于证据摘录，也不等于消息历史
Evidence Store	来源、参数、内容、完整性等	不等于模型已经在本轮看到全部内容
<code>session.messages</code>	协议、目标、响应、工具结果和反馈	不应作为可公开展示的私人推理记录
<code>trace / timeline</code>	事件、阶段、用量与关联标识	不等于完整可续跑的 checkpoint

当前实现怎样约束上下文

当前 `advance` 每轮传入全部 `session.messages` 的拷贝，不会自动摘要旧对话，也不会在上下文太长时从列表里删掉中间消息。控制依靠有限 `step/token/deadline`、工具输入范围与返回素材的规模，`runtime` 还会估算请求 `token` 以检查租户额度。默认案例很小，不能据此保证长日志也能放进任何模型窗口。

案例 `logs` 和 `metrics` 返回全部保留行，工具没有额外裁剪，缺失与采样信息放在 `telemetry` 中。`Runbook` 工具则有 `top_k` 与 `snippet` 限制。`truncated` 是观察被工具截断的标记，缺失数据不等于截断，空数组也不等于所有时刻都没有异常。把这些不同状态写清楚，模型才有机会保留未知项。

源码定位：`src/ailab_ops/tools/cases.py · build_case_registry`

源码定位：`src/ailab_ops/tools/runbooks.py · build_runbook_tool`

若以后加入上下文压缩，应优先保留调查目标、当前计划、尚未排除的假设、已确认事实与证据索引，并保留能重新读取原文的位置。摘要要注明推断与事实的区别，还需保留成对的 `assistant tool_calls` 与 `tool` 结果，避免破坏协议。可压缩的是重复表述，不是证据的来源和限制；这些是扩展设计原则，不是当前功能。

预算控制、上下文窗口管理和知识检索是相互关联的三个问题。更少轮数可降低开销，但不能替代窗口检查；摘要可减少输入，却可能丢失因果细节；检索可补充经验，但不能伪造未采集的本次观察。

章节关联：第 3 课 · 证据对象、完整性与主张—证据校验

2.10 保存、失败与恢复：当前能做到哪一步

保存快照，不保留无限活跃上下文

`_save` 把当前状态拷贝到 `orchestrator.states`，避免历史快照随着之后的字段修改而变化；可选 `persist` 回调得到另一个拷贝。回调失败会记录 `persistence_errors`，不让保存故障直接破坏调查循环。这是一处集成接口，而不是已经完成的数据库事务机制。当前 V2 runtime 未给它接持久数据库。

源码定位：src/ailab_ops/investigation/orchestrator.py · `_save`

`completed` 或 `stopped` 后，控制层归档本会话观察，保留可查证据和业务状态，释放 `sessions` 中的消息、完成调用表和活跃事件。报告引用因此仍可通过 `get_evidence` 解析，其他会话的同名观察不会被混入。这也降低长期保留完整聊天历史的负担，但 `process-local` 数据仍在内存中，服务器重启后会消失。

源码定位：tests/test_orchestrator.py

恢复页面与恢复执行不同

Web UI 保存一个会话标识后，可以在同一服务进程仍保留记录时重新读取状态和证据。这是恢复展示；它不会把 `stopped` 变成 `investigating`，也不会重新启动终止的模型读循环。`advance` 只接受当前 `orchestrator` 管理的 `live session`，并核对状态内容；未知、已释放或陈旧状态抛出 `InvestigationSessionError`。

源码摘录：有一个 `session_id` 不代表能推进任意历史状态

```
session = self.sessions.get(state.session_id)
if session is None or state.to_dict() != session.state.to_dict():
    raise InvestigationSessionError("Unknown, released or stale investigation session")
```

源码定位：src/ailab_ops/investigation/orchestrator.py · `advance`

序列化 `state` 再 `from_dict`，也无法重建已经释放的 `messages`、证据存储和去重表。若未来需要进程重启后续跑，必须设计持久 `checkpoint`：业务状态、模型协议历史、证据、已完成操作、报告修正计数、工具与提示词版本，以及审批和身份关系都要一致恢复。恢复时还必须重新检查期限与权限，不能因为旧请求当时合法，就永久允许执行。

上游失败以后怎么处理

模型重试仍失败时，`orchestrator` 保存 `stopped`，`stop_reason` 标明 `model_backend:timeout` 或 `model_backend:rate_limited` 等分类。`runtime` 展示 `error.kind`、`retryable`、`retry_after_s`。`retryable` 表示此类错误可能在外条件改变后恢复，不等于系统已经安排 `stopped` 自动续跑。用户可检查配置或额度，之后启动新的调查。

在线服务不可用时，用户也可以另开明确标注的 `replay` 调查。它有新的 `session`、预编响应和严格匹配输入，不能被称为延续了刚才的在线判断，也不会对未录制输入伪造响应。当前没有无声把 `online` 改成

replay 的降级路径。恢复的第一步是说明已有观察与停止原因，而不是把界面上的完成状态补漂亮。

源码定位：tests/test_orchestrator.py

2.11 真实运行证据：对照三次预算实验

运行方法与来源

从仓库根目录运行以下已有命令。依赖安装方法见第 1 课；本次无需 GPU 与 API Key。默认 question 保持不变，才能与 authored replay 精确匹配。改预算可以观察同一录制轨迹的前缀；改问题、工具集或回喂内容则可能 replay_miss。

三次已有命令；前两次停止的退出码为 2，第三次 completed 为 0

```
PYTHONPATH=src python3 -m ailab_ops.cli investigate \
  --case case-gpu-assert --mode replay --max-steps 2

PYTHONPATH=src python3 -m ailab_ops.cli investigate \
  --case case-gpu-assert --mode replay --max-tokens 200

PYTHONPATH=src python3 -m ailab_ops.cli investigate \
  --case case-gpu-assert --mode replay
```

以下观测核对于 2026-10-08，代码基线 174f709，数据来自 data/v2/cases/case-gpu-assert.json 与 data/v2/replays/investigations.jsonl。tests/test_course_content.py 会实际重跑三个 CLI 调用，比对所列字段。会话 ID、trace ID、绝对日期与耗时随执行生成，不当作固定结果；本章未使用真实在线模型生成这些测量。

源码定位：data/v2/replays/investigations.jsonl

三次 CLI JSON 的选取摘要；has_report 为报告是否存在的派生字段

```
{
  "steps": {
    "phase": "stopped",
    "stop_reason": "budget_exhausted:steps",
    "steps_used": 2,
    "tokens_used": 400,
    "evidence_count": 3,
    "has_report": false
  },
  "tokens": {
    "phase": "stopped",
    "stop_reason": "budget_exhausted:tokens",
    "steps_used": 1,
    "tokens_used": 200,
    "evidence_count": 0,
    "has_report": false
  },
  "completed": {
    "phase": "completed",
    "stop_reason": null,
    "steps_used": 5,
    "tokens_used": 1000,
    "evidence_count": 4,
    "has_report": true
  }
}
```

先预测，再用轨迹解释

限步实验第一轮更新计划，用 200 个预置 token；第二轮得到三工具请求，再用 200，三个观察都执行并登记。轮内忽略 step 上限，结束时才停，因此 steps_used=2、tokens_used=400、evidence_count=3。后续 Runbook、hypotheses 和 report 都没有执行。日志里已有断言，也不意味着系统发布过结论。

限 token 实验第一轮请求 max_tokens=200，但 replay 返回既定响应与 usage：输入 120、输出 80。累计达到 200 后，程序在处理 plan 前停止，故没有工具、证据或报告。这个例子展示返回后检查的位置；回放不模拟供应商根据 max_tokens 重新生成答案。

默认实验五轮完成，累计 1000 个预置 token，四份证据，report 存在。这些 token 不是在线分词实测或账单，也不能用于比较模型效率。它们适合验证 usage 累加与预算分支。真实在线模式下，更改预算可能改变模型输出和后续路径，不能预期仍然出现完全相同数字。

可追溯检查	观察点	说明
timeline 中 model requested/received	第一实验各两次；第二实验各一次	开始决策与收到响应分别记事件
tool result / evidence captured	第一实验三次；第二实验零次	证据来自实际执行的读取
末尾 state stopped	stop_reason 与 budget 同时保存	停止有原因与已用量
完整实验 validating_report / completed	报告校验后完成	不能仅数工具调用判断任务完成

错误反例如何核对

默认录制轨迹没有包含参数错误、重复调用或上游故障。要核对这些反例，可阅读并运行仓库既有测试：它们使用脚本化模型边界与真实控制层、工具、证据对象，检查错误如何进入下一轮及实际副作用。错误反例在本章标为结构示例或设计推演，不添加到正常回放的五轮测量里。

课后可运行的既有控制层与网关回归测试

```
PYTHONPATH=src python3 -m pytest tests/test_orchestrator.py -q
PYTHONPATH=src python3 -m pytest tests/test_model_backends.py tests/test_model_controls.py -q
```

源码定位：tests/test_model_backends.py

源码定位：tests/test_model_controls.py

2.12 常见错误与本章总结

从失败模式倒推保护位置

反例	错误直觉	本项目对应保护与限制
重复读取同一日志	再次调用就会增加确信	规范化键阻止成功重复读取；仍消耗模型预算
编造 case_id 或参数名	程序猜到意图后帮它改对	schema/策略拒绝并回喂；不静默改写
一直返回新计划，无法停止	要求模型最后一定结束即可	总预算强制停止；plan 不赋予执行权
模型限流后无限等待	Retry-After 必须照单全收	有限尝试、等待上限、取消和截止复查
慢工具越过截止时间	循环有 deadline 就会立刻中断	当前同步工具只能返回后检查；远程适配需超时
保存 state 即可重启续跑	JSON 是完整 checkpoint	state 只是业务记录；会话历史与执行表也必需

一次可控调查必须同时回答选择与边界：模型决定下一步看什么，程序决定这步能否开始、结果是否有效、何时必须结束。phase 让过程可读，Budget 让开销有终点，结构化错误让修正可追踪，证据索引让报告可查，有限报告修正避免校验变成新一轮无限调查。每个保护都应找到实际执行它的代码位置。

本案例中的 GPU assert 结论不由状态机选出。状态机能拒绝重复读取、错误参数和无效引用，却不能保证模型一定提出正确假设；一个程序上合法的报告仍可能在语义上误判。下一章将继续讨论观察、知识

与证据如何支撑回答，第 5 课再用独立评测判断实际质量。

课后阅读与模仿方向

1. 从 investigation/models.py 画出业务状态字段，再对照 InvestigationSession，解释为何 state 与运行上下文分开。
2. 沿 advance、_stop_if_exhausted、_read_tool、_control、_report_issues 阅读，给每个提前返回写一句原因。
3. 运行本章三条回放命令，记录模型轮数、工具次数、证据增量和报告是否存在；对照真实 timeline。
4. 阅读 tests/test_orchestrator.py 的重复、参数错误、报告修正和部分状态测试，区分模型边界替身与真实控制程序。
5. 课后在临时分支模仿一个小任务：先定义状态字段、成功条件、停止原因与输入错误结果，再连接真实模型。

模仿时可以只保留两种读取与一种报告，不必复刻全项目。先写下哪些结果能进入证据、同一次请求如何识别、工具失败后是重试还是换方向，以及运行结束保留什么。若选择离线演示，明确称作人工编写回放；若选择在线模型，保存实际轨迹并核对行为，不把书中的预置数字当作目标成绩。

思考题与校对线索

问题	校对线索
max_steps=2 为什么能执行三个工具？	step 计模型决策；最后一轮允许批量读取，下一轮才被阻止。
输入加输出超过剩余 token 怎么办？	预算在 response usage 返回后检查；请求 max_tokens 主要限制输出。
duplicate_call 返回 ok=false，是否意味着没有既有证据？	已有 evidence_ids 随错误返回；本次没有新增读取。
一次工具失败与一次报告失败的修正次数一样吗？	前者由总预算约束，后者最多一次报告修正。
限流 retryable=true 能否证明会话正在自动恢复？	它是错误分类；stopped 没有自动续跑实现。
若第一工具执行期间超时，其观察是否丢弃？	同步返回的有效观察保留，随后阻止后续批次。
恢复页面后为什么不能推进历史 completed？	终态已释放 live session；状态快照与消息上下文不同。

理解控制之后，阅读下一章时可以继续追问：同一份观察为什么成为一条证据，Runbook 给的是经验还是事实，以及引用存在为什么仍不足以证明结论。控制层给调查划出边界，证据层帮助我们检查边界内的推断。